Modeling: MultiModal AI — Homework 3

MAS.S60 / 6.S985 • Spring 2026 • MIT

In this homework, you will explore Vision-Language Models (VLMs) and gain hands-on experience fine-tuning one.

Environment Setup

Go to the top menu:
Runtime → Change runtime type → Hardware accelerator → Choose "A100"

If you do not have Colab Pro, you can sign up for a free student Colab Pro account here:
https://colab.research.google.com/signup

Part 1: Reading & Reflection (20 points)

Required Reading

Multimodal Few-Shot Learning with Frozen Language Models

Quality Not Quantity: On the Interaction between Datase Design and Robustness of CLIP

Generative AI: Here to stay, but for good?

Questions

1. What types of multimodal data noise are typically present in multimodal datasets, and how can they negatively impact the performance of a model during training? Can you provide examples of multimodal data points that might be considered noisy? Furthermore, how might we develop estimators capable of distinguishing between noisy and noise-free multimodal data pairs? If you have unlimited fundings to use for data filtering and data cleaning, what would be the ideal way to clean the multimodal dataset?

2. What is the intuition of utilizing frozen large language models as the backbone for multimodal tasks? Which types of encoders would facilitate the integration of diverse information into a format understandable by LLMs? How do these LLMs process and interpret information from different modalities?

3. Ensuring the effectiveness of multimodal foundation models through high-quality instruction tuning is vital. A study detailed at here introduces a strategy for selecting significant data specifically suited for enhancing instruction tuning for language models. A primary challenge in this approach is determining which data are most crucial for targeted instruction tuning. How can we accurately identify and select the most impactful data for enhancing instruction tuning in multimodal foundation models? Given the

complexity of diverse and multimodal information, what strategies can ensure the effectiveness of instruction tuning data for specific tasks?

4. With the advancement of generative AI, distinguishing between AI-generated and human-created content is becoming increasingly challenging. Besides watermarking, which has its limitations, are there other effective methods to differentiate between AI-generated and human-created content across various modalities (text, audio, video, image)? Or is it becoming virtually impossible to make this distinction?

5. For state-of-the-art video generation models like Sora, Yann Lecun mentioned in here that Sora does not understand the real world and its corresponding physical rules. Do you agree with this view? Can the future development of generative AI systems truly incorporate real-world knowledge, or are they limited in this aspect? Is pursuing generative AI a viable path towards achieving Artificial General Intelligence (AGI)?

Answers:

1. First, any type of unimodal noise can be a problem. As in *Quality Not Quantity*, common issues include adversarial examples or corruptions (Gaussian noise or blur for images). Other examples for vision include defocus/bokeh, JPEG/compression/digitizing artifacts, motion blur, while examples for text include typos, false cognates, poor word choice, etc. Obviously, such unimodal noise can reduce modality encoder accuracy.

A more common and insidious case: multimodal noise, where the corruption is not limited to a single modality. Notice equivalent corruption to both modalities is generally a non-issue; methods like ambient diffusion and learning from noisy data can still align and fuse correctly (though of course accuracy is worse than with clean data). Even if one modality is corrupted and the other clean, the model may be able to recover clean internal representations with sufficient data (central limit theorem). Unfortunately, most often the corruption in one modality is paired with a different corruption in the other modality---which, like the curse of dimensionality, can scale exponentially with the number of modalities (making central limit theorem difficult to apply). An example might include a blurred image with an incorrect (not simply vague) caption.

One approach (from *Quality Not Quantity*) considers alignment. Essentially, models pretrained on individual modalities should be most invariant to a change in modality (a distribution shift). So the cleanest data under this model is the data with the most consistent information across modalities (and of course smaller sub-distributions to ensure sufficient sampling).

Given unlimited funding/infinite compute, this translates to filtering each modality with a teacher model, which estimates alignment for different samples (and subselects for higher alignment). Then the student model can be pretrained on the high-alignment subset (so it has a cleaner curriculum to learn), after which it replaces the teacher model and we iterate. If using an ensemble of models for different modalities, the ensemble output is a predictor of the alignment and thus the lack of noise in the dataset---so this can be used for a stopping criterion.

One consideration: repeated subsampling may decrease overall training-set diversity; we want maximum information (mutual information = alignment in this case) for a particular fixed variance (training set diversity), so care must be taken to retain diverse samples.

2. From the *Frozen* paper, fine-tuning the languange model is risky due to data imbalance; the fine-tuning dataset (paired image-caption data) is much smaller than the original text corpus. Further, an off-the-shelf language model is readily available, modular, and somewhat generic; LLMs are somewhat general-

purpose computing in modern ML. To encode, tokenizers are quite common. Just train an encoder and quantize to a discrete token (preferably with a text-contrastive approach like CLIP, to align to text), and the LLM will be able to process it. One disadvantage of current LLMs (rapidly changing with discrete-diffusion) includes the autoregressive generation, as this imposes a sequential dependence among the tokens. On the other hand, we can do simple pattern-matching / online adaptation relatively easily with the autoregressive structure (just provide paired examples as prior tokens).

3. The *LESS* approach selects samples with gradients from a warmup LORA training that most match those from validation data. More generally, we want samples with the maximum new information; *LESS* just estimates the score (grad log p, or equivalently here the active subspace) using the validation distribution. There are a lot of similar methods from general ML, information theory, and reduced order modeling community: some examples would be optimal experiment design (OED, via Bayes law), community detection with the strong data-processing-inequality (SDPI)...but most of them rely on some approximation to the score. For multimodal models, one approach would essentially take the approach in #1 for cleaning data, but instead of alignment across distributions, we would require alignment to either the target distribution or a high-quality teacher model.

Of course, a simple approach would involve just using CLIP or other contrastive encoders (or even just PCA across multiple modalities) to provide a cheap, approximate score directly.

Some strategies that help ensure the effectiveness of instruction tuning: as in *Frozen*, make sure the multimodal tokenizers all align with the instruction and text token format (and align semantically as well). Few-shot (or at least one-shot) prompting also helps the model understand the instruction, leveraging the autoregressive structure.

4. Actually a rather simple question to answer, as we just want to measure the difference between the generated and true distributions. There are a lot of ways to do this, and a lot of ways to fail too. For example, as in *Generative AI*, an enduring difference is the limited (likely historical) information given to the generative model: assuming the training distribution is known, community detection can be used to identify true images (which are out-of-distribution with respect to the generative / training distribution). Some examples of this could include historical biases present in current datasets. Recent work by Adam Klivans and others suggest that simply computing the different moments of each distribution with limited samples can allow you to separate them. Of course, a per-sample basis can still be indistinguishable, as it is possible for even a badly-trained model to just memorize a training sample. And currently, at least for images, we know the perception/fidelity metrics are flawed (FID) and incomplete.

5. As always, there are arguments for both directions. Simply from the bitter lesson/scaling arguments, we know that with enough data, a large enough model, and the right loss/optimizer combination, even a simple generative model can become a world model. This is not just true for ML models; even physical (PDE) equations can be reformulated as minimization objectives for the action (the Lagrangian action). With enough data, the right loss will provide some supervision on the action, so the model will converge to true physics as best possible for the architecture (modulo optimizer, of course). On the other hand, there's always an undersampling problem with such a formulation. To avoid this, we add inductive biases, so the model doesn't just memorize some well-sampled regions. Arguments such as LeCun's fall in this category; they say we require better representations that create the right manifold (essentially construct the right inductive biases). For scaling/gen-ai groups, AGI is actually a perpetual improvement as they keep trying to better sample reality. It's similarly possible that the right manifold (for everything) doesn't exist in simple closed form (i.e. easily representable with an ML model).

Part 2: Testing and Fine-tuning VLMs (100 points)

Problem 1: GPU Verification and Library Installation

Run the following code cell to verify that your environment is correctly configured.

This step ensures that **PyTorch** and **CUDA** can access the GPU.
When the setup is correct, a **secret word** will appear in the output.

In Your PDF Submission

Include:

- A **screenshot** or **code snippet** showing the printed GPU information.
- The **secret word** displayed by your verification cell.

In [1]:
```
!uv pip install transformers accelerate bitsandbytes pillow torch -q
```

In [1]:
```
import torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("CUDA device count:", torch.cuda.device_count())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
t = torch.randn(2, 3, device=device)
KEY = 42
cipher_bytes = [99, 10, 102, 101, 124, 111, 10, 103, 103, 107, 99]

if t.is_cuda:
    cipher = torch.tensor(cipher_bytes, dtype=torch.uint8, device=device)
    decoded = torch.bitwise_xor(cipher, KEY)
    torch.cuda.synchronize()
    secret = bytes(decoded.tolist()).decode("ascii")
    print("SECRET_WORD:", secret)
else:
    print("SECRET_WORD: (not on GPU)")
```

```
PyTorch version: 2.10.0+cu128
CUDA available: True
CUDA device count: 1
GPU name: NVIDIA L40S
SECRET_WORD: I LOVE MMAI
```

Problem 2: Prepare Your Dataset (20 points)

PLEASE READ THIS ENTIRE SECTION BEFORE PROCEDING

For Problem 2, you will **use the dataset you have collected from Homework 1 and Homework 2 or a completely new one if you prefer** to fine-tune a Vision-Language Model (VLM).

Even if your original data isn't image-based (e.g. it's audio, time-series, or text), you should find a way to **visualize it** meaningfully. The dataset you prepare will serve as the foundation for model fine-tuning in later steps.

How to Convert Your Project Data Into Images

If your project is not originally image-based, consider these ideas to generate visual input:

Data Type	Visual Representation Example
Time-series / sensor data	Line plots or multi-panel charts (with axis labels)
Audio / Music / Physiology	Spectrograms or waveform plots

| 3d data (point clouds, CAD) | Rendering/splicing into 2D images

You are encouraged to be **creative and domain-specific** in your visualizations.

You will need to explore ways to convert your data into images if it does not already consist of this modality. Research on your own and come up with the needed code to do so. If you are still stuck on figuring this out, please reach out to a TA for help!

Download Example Training Data

The next block of code will download an example dataset and create a folder named `mmai-data/`.
Inside this folder, you will find:

```
mmai-data/
├── images/
│   ├── 1.jpg
│   └── 2.jpg
└── data.jsonl
```

The file `data.jsonl` contains your training annotations.
Each line represents one training example with the following fields:

```
{
  "image": "images/1.jpg",
  "question": "List objects you see.",
  "answer": "cat, sofa, blanket, remote, cushion"
}
```

Your Task

Now, prepare your own dataset following the same structure as the example.

Example structure:

```
mmai-data/
├── images/
│   ├── image_01.jpg
│   ├── image_02.jpg
│   ├── ...
└── data.jsonl
```

As part of this task. You should split the data into a train and test split. **The test split should consist of the images of data that you will not use in training.**

Answer: We will try to answer two questions together --- what is the absolute simulation time of the image (i.e. how far from the Gaussian noise initialization is it) and what is the local intrinsic dimensionality of the manifold (computed using the TwoNN method below).

In [2]:
```
from torch.utils.data import DataLoader, random_split
%matplotlib inline
import matplotlib.pyplot as plt
import matplotlib
backend = matplotlib.get_backend()
import jpcm.draw as draw
matplotlib.use(backend)
from matplotlib.colors import TwoSlopeNorm
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch_dct as dct
from tqdm import tqdm
```

In [3]:
```
data = '/orcd/home/002/a1744874/orcd/pool/ml/datasets/forced_turbulence/v1_20260216_8ec01795/forc
dataset = torch.from_numpy(np.load(data, mmap_mode='r')[:,1:-2,:1,:,:])
print(dataset.shape)
```

```
torch.Size([20, 498, 1, 512, 512])
```

In [4]:
```
import torch

def intrinsic_dimension_twoNN(data, chunk=498, downsample=1):
    """
    data shape: (B, T, 1, H, W)

    returns:
        dims: (B, T)
    """

    # optional spatial downsample
    data = data[...,::downsample, ::downsample]
    B, T = data.shape[:2]
    dims_all = torch.zeros(B,T,device=data.device)
    for ti in tqdm(range(data.shape[1] // chunk)):
        t = ti * chunk
        i0 = max(t - chunk, 0)
        shift = t - i0
        i1 = min(t + 2*chunk, data.shape[1])
        chunk_data = data[:, i0:i1, :]

        B, T, C, H, W = chunk_data.shape
        D = C * H * W
```

```
        # flatten spatial
        X = chunk_data.reshape(B, T, D).to('cuda')

        # pairwise distances across time
        dot = torch.bmm(X, X.mT)
        norm = (X**2).sum(-1, keepdim=True)

        dists = norm + norm.mT - 2*dot
        dists = torch.sqrt(torch.clamp(dists, min=1e-12))

        # get two nearest neighbors
        vals, _ = torch.topk(dists[:,shift:shift + chunk], k=3, largest=False)

        r1 = vals[:, :, 1]
        r2 = vals[:, :, 2]

        ratio = r2 / r1

        dims = 1.0 / torch.log(ratio)
        dims_all[:,chunk * ti: chunk * (ti + 1)] = dims
    return dims_all
```

In [5]:
```
dims = intrinsic_dimension_twoNN(dataset, downsample=1)
```

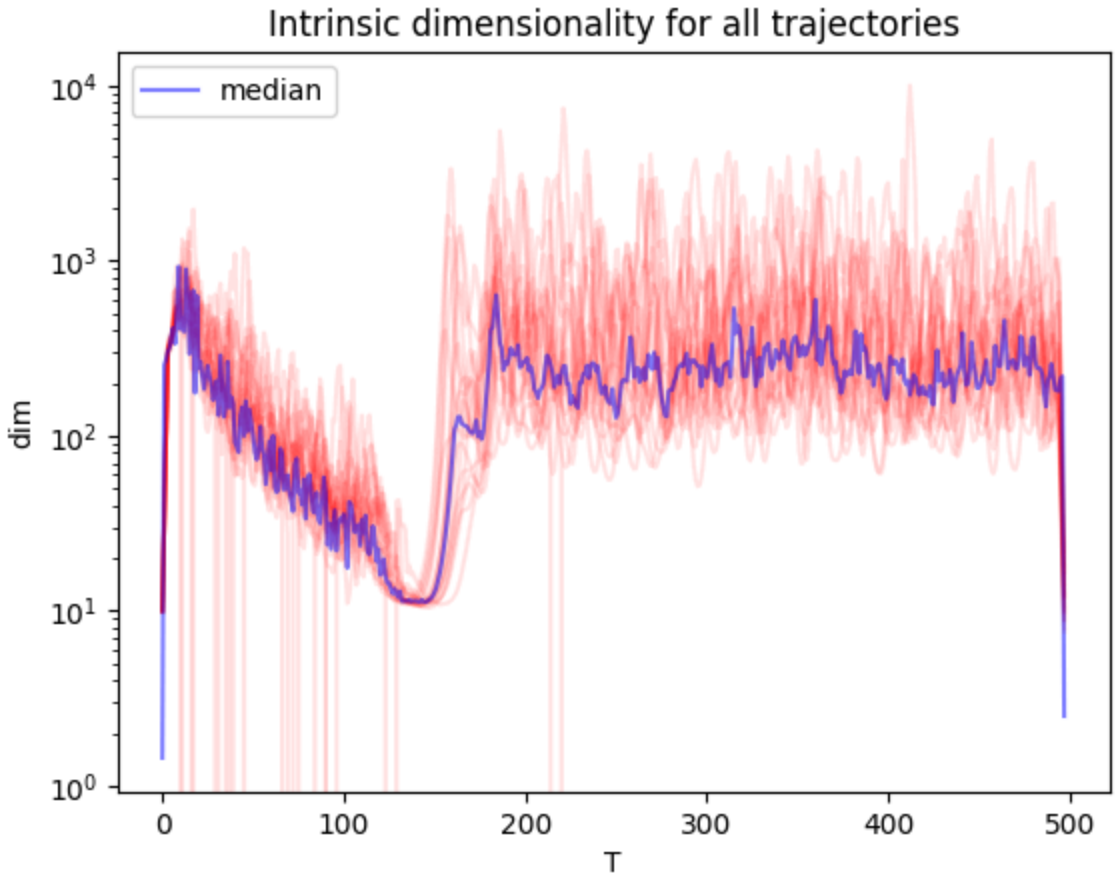
```
100%|██████████████████████████████████████████████████████████| 1/1
[00:03<00:00,  3.52s/it]
```

In [6]:
```
def smooth(data, window_size=5):
    padding = window_size // 2
    sm = F.avg_pool1d(data, kernel_size=window_size, stride=1, padding=padding)
    qm = F.avg_pool1d((data > 0).to(int), kernel_size=window_size, stride=1, padding=padding)
    qm[qm <= 0] = 1
    assert sm.shape == data.shape
    return sm/qm
```

In [7]:
```
# smooth in x first
dims[~torch.isfinite(dims)] = 0
sdims = smooth(dims.log()).exp()
# plt.semilogy(sdims[0])
# plt.show()

assert torch.isfinite(sdims).all()

for i in range(len(dims)):
    d = sdims[i,:]
    # d[d>50000] = 0
    plt.semilogy(d, c='r', alpha=0.1)
plt.semilogy(dims.log().median(0)[0].exp(), c='b', alpha=0.5, label='median')
plt.title("Intrinsic dimensionality for all trajectories")
plt.xlabel("T")
plt.ylabel("dim")
plt.legend()
plt.show()
```



Above is the local intrinsic dimensionality of the data manifold; this is one requested output of the LLM (to test if it can use the features to identify at least the size of the space required for representation).

In [8]:
```
T=(torch.arange(dataset.shape[1]) + 1)[None,:].expand(*dataset.shape[:2])
print(T.shape, sdims.shape, dataset.shape)
```

```
torch.Size([20, 498]) torch.Size([20, 498]) torch.Size([20, 498, 1, 512, 512])
```

In [9]:
```
label=lambda t, sdim: f'{t}, {int(round(sdim.item()))}'
```

In [23]:
```
import os
import json
import torch
from torchvision.utils import save_image
from tqdm import tqdm

root = "mmai-data"
img_dir = os.path.join(root, "images")

os.makedirs(img_dir, exist_ok=True)

question = (
    "How far from an initial-condition of Gaussian noise is this flow field? "
    "What is the intrinsic dimensionality of the field? "
    "Note that the system reaches self-similarity at T=175 and the manifold "
    "dimension is then close to 32 thereafter. "
    "Format your answer as the following example (time and manifold dim) "
    "respectively: 175, 32."
)

jsonl_path = os.path.join(root, "data.jsonl")

idx = 0
```

```python
with open(jsonl_path, "w") as f:

    for b in tqdm(range(dataset.shape[0]-1)): # skip last batch, we will use that for test.
        for t in range(dataset.shape[1]):

            img = dataset[b, t]

            # normalize for visualization
            img = img - img.min()
            img = img / (img.max() + 1e-8)
            # img = draw.cmap(img)

            img_path = f"images/image_{idx:06d}.jpg"
            full_img_path = os.path.join(root, img_path)

            save_image(img, full_img_path)

            answer = label(T[b, t], sdims[b, t])

            item = {
                "image": img_path,
                "question": question,
                "answer": answer
            }

            f.write(json.dumps(item) + "\n")

            idx += 1

print("Dataset written to:", root)
```

```
100%|██████████████████████████████████████████████████████████| 19/19
[00:45<00:00,  2.42s/it]
Dataset written to: mmai-data
```

In [10]:

```python
import os
import json
import torch
from torchvision.utils import save_image
from tqdm import tqdm

root = "mmai-data-test"
img_dir = os.path.join(root, "images")

os.makedirs(img_dir, exist_ok=True)

json_path = os.path.join(root, "data.json")
jsonl_path = os.path.join(root, "data.jsonl")
```

In [11]:

```python
question = (
    "How far from an initial-condition of Gaussian noise is this flow field? "
    "What is the intrinsic dimensionality of the field? "
    "Note that the system reaches self-similarity at T=175 and the manifold "
    "dimension is then close to 32 thereafter. "
    "Format your answer as the following example (time and manifold dim) "
    "respectively: 175, 32."
)
```

```python
idx = 0

items = []
with open(jsonl_path, "w") as f:

    # for b in tqdm(range(dataset.shape[0]-1)): # skip last batch, we will use that for test.
    b = dataset.shape[0] - 1
    for t in range(dataset.shape[1]):

        img = dataset[b, t]

        # normalize for visualization
        img = img - img.min()
        img = img / (img.max() + 1e-8)
        # img = draw.cmap(img)

        img_path = f"images/image_{idx:06d}.jpg"
        full_img_path = os.path.join(root, img_path)

        save_image(img, full_img_path)

        answer = label(T[b, t], sdims[b, t])

        item = {
            "image": img_path,
            "question": question,
            "answer": answer
        }

        f.write(json.dumps(item) + "\n")
        items.append(item)
        idx += 1

with open(json_path, 'w') as json_file:
    json.dump(items, json_figle, indent=4)

print("Dataset written to:", root)
```

```
Dataset written to: mmai-data-test
```

In [12]:
```python
test_items = items
```

Questions to Answer:

- Explain some possible issues with converting non-image data into images (even if you did not have to do so, discuss what could be some issues).
- What are some possible issues with using visual representations of your data. Discuss some drawbacks of doing this (if you did not have to do the conversion as your data was already in the form of images, then discuss the drawbacks of converting those images to another modality like text, audio, etc.).
- Discuss the strategy you decided on how to split your data into train/test splits. Why did you settle on this? Were any other alternative splits considered?

Answer:

1. Take PDE data, in particular the matrix representation of some groups. By group theory, all groups can be represented as a subset of a symmetry group (over all elements, very large) which can be

represented as a permutation matrix. So these matrix representations can then be rendered as images. Unfortunately, these images can be infinitely large (as are the matrix representations). A similar issue exists for smooth continuous flow fields; making an image is equivalent to forcing a regular discretization, which can introduce artifacts or incorrect boundary conditions.

2. Take audio data. To make a spectrogram, you need to choose an STFT window, which enforces a particular discretization of your data, which may be inaccurate depending on the signal and sparsity (window size). Another issue: most image encoders are x-y convolutional/shift-invariant in nature (unless you have strong positional encoding and a ViT or DiT architecture), which can be quite different --- for example, audio spectrograms have a particular directionality (time vs pitch), and for the group representation case, CNNs might not respect matrix invertibility.

3. I just left out one simulation run as a test split. In general we have too little data, so that felt most natural, and each simulation is a IID sample, so in the limit of many simulations, there is no distribution shift. Lastly, there is a large variation along each simulation in time, so it's already a hard problem for the LLM. No other splits were considered.

Problem 3: Baseline Inference (10 points)

Problem 3.1 Load the Model

Begin by running the following code to **load the base model** into memory. This step is required before training or making predictions.

In [185…

```
del model
del processor
import gc
gc.collect()
torch.cuda.empty_cache()
```

In [13]:

```
import io, requests, torch
from PIL import Image, UnidentifiedImageError
from transformers import AutoProcessor, Qwen2_5_VLForConditionalGeneration

model_id = "Qwen/Qwen2.5-VL-3B-Instruct"

# 1) Load model + processor (processor handles BOTH text + vision)
processor = AutoProcessor.from_pretrained(model_id)
model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
    model_id,
    dtype=torch.bfloat16 if torch.cuda.is_available() else torch.float32,
    device_map="auto" if torch.cuda.is_available() else None,
)
print("Model and tokenizer loaded successfully.")
```

```
The image processor of type `Qwen2VLImageProcessor` is now loaded as a fast processor by default,
even if the model checkpoint was saved with a slow processor. This is a breaking change and may p
roduce slightly different outputs. To continue using the slow processor, instantiate this class w
ith `use_fast=False`.
Fetching 2 files: 100%|██████████████████████████████████████████████████████████| 2/2 [0
0:00<00:00, 206.60it/s]
Loading weights: 100%|└└└
```

Image ID/URL	Question	Model Output	Ground Truth	Result
`img_001.jpg`	"What objects are visible?"	cat, sofa	cat, sofa	Correct
`img_002.jpg`	"What objects are visible?"	road, truck, sign	road, car, sign	Incorrect

In [17]:
```
import io
import os
import requests
import torch
from typing import Optional, Dict, Any
from PIL import Image, UnidentifiedImageError
from transformers import AutoProcessor, Qwen2_5_VLForConditionalGeneration


# ============================================================
# ######################### CHANGE ME #########################
# ============================================================
IMAGE_URL: str = "http://images.cocodataset.org/val2017/000000039769.jpg"
QUESTION: str = question #"How does the image make you feel? Show a list of emotion labels only.
MAX_NEW_TOKENS: int = 512 #128
# ============================================================
# ######################## END CHANGE ME ######################
# ============================================================


# SYSTEM CONFIG
DO_SAMPLE: bool = False           # set True for non-greedy decoding
TEMPERATURE: float = 0.7          # used only if DO_SAMPLE=True
TOP_P: float = 0.9                # used only if DO_SAMPLE=True
MODEL_ID: str = "Qwen/Qwen2.5-VL-3B-Instruct"
FORCE_CPU: bool = False           # force CPU even if CUDA is available
DTYPE_IF_GPU = torch.bfloat16     # prefer bfloat16 on recent GPUs/Colab
DTYPE_IF_CPU = torch.float32

def get_device_and_dtype() -> tuple[torch.device, torch.dtype, Optional[Dict[str, Any]]]:
    """Choose device/dtype and (optionally) a device_map for accelerate-style placement."""
    use_cuda = torch.cuda.is_available() and not FORCE_CPU
    device = torch.device("cuda") if use_cuda else torch.device("cpu")
    torch_dtype = DTYPE_IF_GPU if use_cuda else DTYPE_IF_CPU
    device_map = "auto" if use_cuda else None
    return device, torch_dtype, device_map


def load_image_from_url(url: str) -> Image.Image:
    """Fetch image from URL and return a RGB PIL.Image with robust fallback."""
    resp = requests.get(url, timeout=30)
    resp.raise_for_status()
    try:
        return Image.open(io.BytesIO(resp.content)).convert("RGB")
    except UnidentifiedImageError:
        # Fallback: write to disk then reopen (sometimes fixes truncated headers)
        tmp_path = "temp_image.jpg"
        with open(tmp_path, "wb") as f:
            f.write(resp.content)
        img = Image.open(tmp_path).convert("RGB")
        try:
            os.remove(tmp_path)
        except Exception:
            pass
        return img
```

```python
def load_from_test(n=4):
    l = len(test_items)
    idx = np.linspace(start=0, stop=l-1, num=n+1)[1:].astype(int)
    # print(test_items)
    select = [test_items[i] for i in idx]
    images = [Image.open('mmai-data-test/' + s['image']).convert("RGB") for s in select]
    paths = ['mmai-data-test/' + s['image'] for s in select]
    answers = [s["answer"] for s in select]
    return zip(paths, images, answers)

def build_chat_messages(SYSTEM_PROMPT, image: Image.Image, question: str) -> list[dict]:
    """Create a single-turn, image+text chat for Qwen-VL processors."""
    return [
        {
            "role": "system",
            "content": [
                {"type": "text", "text": SYSTEM_PROMPT}
            ],
        },
        {
            "role": "user",
            "content": [
                {"type": "image", "image": image},
                {"type": "text", "text": question},
            ],
        }
    ], []


def main(model=None, processor=None, SYSTEM_PROMPT: str = "You are a helpful assistant.", _build_
    device, torch_dtype, device_map = get_device_and_dtype()

    # 1) Load model + processor
    processor = AutoProcessor.from_pretrained(MODEL_ID)
    if model is None:
        model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
            MODEL_ID,
            torch_dtype=torch_dtype,
            device_map=device_map,
        )
        print("Model and processor loaded successfully.")

    test_results = []

    for path, image, true_msg in load_from_test(6):

        # 2) Load image
        # image = load_image_from_url(IMAGE_URL)

        # 3) Build chat
        messages, images = _build_chat_messages(SYSTEM_PROMPT, image, QUESTION)

        # 4) Apply chat template and preprocess
        text = processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True
        inputs = processor(text=[text], images=[*images, image], return_tensors="pt")
        # print(inputs)

        # 5) Move to the right device
        inputs = {k: v.to(model.device) for k, v in inputs.items()}
```

```python
        # 6) Generate
        gen_kwargs = dict(max_new_tokens=MAX_NEW_TOKENS)
        if DO_SAMPLE:
            gen_kwargs.update(dict(do_sample=True, temperature=TEMPERATURE, top_p=TOP_P))

        with torch.no_grad():
            gen_ids = model.generate(**inputs, **gen_kwargs)

        # 7) Decode
        out = processor.batch_decode(gen_ids, skip_special_tokens=True)[0]
        # print("\n=== MODEL OUTPUT ===")
        # print(out)
        # print(true_msg)
        full_response = out.split('assistant')[-1]
        response = full_response.split('\n')[-1].strip('\n').strip('.')
        # print(out)

        def error(a, b):
            at, ad = np.array(a.split(',')).astype(int)
            bt, bd = np.array(b.split(',')).astype(int)
            return at-bt,np.log(ad/bd)


        test_results.append([path, response, true_msg, *error(response,true_msg), full_response]

    import pandas as pd
    import base64, io
    from IPython.display import display, HTML

    rows = []

    for image, response, true_msg, err1, err2, resp in test_results:
        img_html = f'<img src="{image}" width="80">'

        rows.append({
            "image": img_html,
            "response": response,
            "true": true_msg,
            "time-error (in frames)": err1,
            "dimension error (log of ratio)": err2,
            "reasoning": resp
        })

    df = pd.DataFrame(rows)
    display(HTML(df.to_html(escape=False)))
```

In [70]:

```python
if __name__ == "__main__":
    main()
```

```
Fetching 2 files: 100%|██████████████████████████████████████████████████| 2/2 [00:
00<00:00, 19972.88it/s]
Loading weights: 100%|└└└
```

```
Model and processor loaded successfully.

175, 32

175, 32

175, 32

175, 32

175, 32

175, 32
```

		image	response	true	time-error (in frames)	dimension error (log of ratio)	reasoning
0		No description has been provided for this image	175, 32	83, 42	92	-0.271934	\n175, 32
1		No description has been provided for this image	175, 32	166, 16	9	0.693147	\n175, 32
2		No description has been provided for this image	175, 32	249, 260	-74	-2.094946	\n175, 32
3		No description has been provided for this image	175, 32	332, 469	-157	-2.684867	\n175, 32
4		No description has been provided for this image	175, 32	415, 418	-240	-2.569746	\n175, 32
5		No description has been provided for this image	175, 32	498, 10	-323	1.163151	\n175, 32

Clearly the pretrained model has just about no clue what I'm asking for.

Ans: What worked, failed and why.

Independent of basically everything I tried, the model doesn't generalize at all. The task is rather hard, so the model just defaults to the prior I have given (simulation time is 175 and dimension is 32) instead of estimating something based on the image. Part of the issue is likely while a human has a prior of what flow/ocean vorticity fields look like, the model has little-to-none of similar data in the original training dataset, so it can't even look for the right features.

Problem 4: Prompt Engineering (15 points)

In this step, you'll experiment with **prompt design** to explore how different instructions influence model performance.

Instructions

1. Modify the `SYSTEM_PROMPT` variable inside the **CHANGE ME** section of the code above.
2. Re-run the corresponding code cell to observe how the model's responses change.
3. Test various prompt strategies, such as:
   - Adding **examples** (few-shot prompting)

- Restricting **answer formats** (e.g., "Answer with one word")
- Asking for **explanations** or **step-by-step reasoning**

4. Compare your new results with the baseline output.

Reflection

In your write-up, discuss:

- Which types of prompt changes improved performance?
- Did adding context or structure help the model reason more effectively?
- Were there any surprising or inconsistent results?

In [109…

```
# let's start with reasoning and hint more
main("""We are analyzing the dynamics of quasigeostrophic flow.
The available flow fields have time from 0 to 500 with intrinsic manifold dimension on a similar
In general, smooth and regular fields are closer to the Gaussian initialization (time 0) and more
Use your expertise in ocean and atmospheric flows to reason about the structures of the flow fie
The answer is not complete without some short reasoning, and you must restate the final values o
in the user's specified format of number, number on a new line at the end.""")
```

```
Fetching 2 files: 100%|██████████████████████████████████████████████████| 2/2 [0
0:00<00:00, 206.68it/s]
Loading weights: 100%|└└└
```

	image	response	true	time-error (in frames)	dimension error (log of ratio)	reasoning
0	No description has been provided for this image	175, 32	83, 42	92	-0.271934	\nGiven the information provided, it seems that the flow field is evolving towards a state closer to the Gaussian initialization by time 175, where the intrinsic dimensionality is approximately 32. Therefore, the answer is:\n\n175, 32.
1	No description has been provided for this image	175, 32	166, 16	9	0.693147	\nThis flow field appears to be quite turbulent, suggesting it is further away from the Gaussian initialization than the smoother fields observed around time 0. Given the information provided, we can infer that this field is likely closer to the later times when turbulence is more pronounced.\n\nBased on the description, the intrinsic dimensionality of the field is approximately 32, which aligns with the observation that the system reaches self-similarity at $T = 175$.\n\nTherefore, the answer is:\n175, 32.
2	No description has been provided for this image	400, 32	249, 260	151	-2.094946	\n400, 32
3	No description has been provided for this image	400, 32	332, 469	68	-2.684867	\n400, 32
4	No description has been provided for this image	500, 32	415, 418	85	-2.569746	\n500, 32
5	No description has been provided for this image	400, 32	498, 10	-98	1.163151	\n400, 32

Ans: Reasoning is much better. At least now, with the expanded prompt, it can identify the right range and at least the spin-up phase (when the flow-field isn't fully developed with the right frequency spectrum). Of course, the times are still quite off, notably it cannot tell where it is after spinup, nor anything about the dimensionality.

In [20]:
```
def load_from_train(n=5):
    lines = open('mmai-data/data.jsonl', "r").readlines()
    idx = np.linspace(start=0, stop=len(lines)-1, num=n).astype(int)
    lines = [lines[i] for i in idx]
    select = [json.loads(l) for l in lines]

    images = [Image.open('mmai-data/' + s['image']).convert("RGB") for s in select]
    answers = [s["answer"] for s in select]

    return images, answers
```

```python
def build_chat_messages2(SYSTEM_PROMPT, image: Image.Image, question: str):
    N = 48
    images, answers = load_from_train(N)
    ind = torch.randperm(N)
    images = [images[i] for i in ind]
    answers = [answers[i] for i in ind]
    messages = [
        {
            "role": "system",
            "content": [{"type": "text", "text": SYSTEM_PROMPT}],
        }
    ]

    # few-shot examples
    for im, ans in zip(images, answers):

        messages.append({
            "role": "user",
            "content": [
                {"type": "image", "image": im},
                {"type": "text", "text": question},
            ],
        })

        messages.append({
            "role": "assistant",
            "content": [{"type": "text", "text": ans}],
        })

    # actual query
    messages.append({
        "role": "user",
        "content": [
            {"type": "image", "image": image},
            {"type": "text", "text": question},
        ],
    })

    return messages, images
```

In [184…

```python
main("""We are analyzing quasigeostrophic turbulence.
Early times:
smooth flow with few vortices.
Middle times:
vortex merging and filament formation.
Late times:
many thin filaments and small vortices.
The flow eventually becomes self-similar and the intrinsic
manifold dimension saturates.
Use the visual structure of the field to estimate the stage
of turbulence; provide your best guess. Use previous images to inform.""", _build_chat_messages=l
# used GPT assistance to get a better prompt after trying a couple that didn't work well
```

```
Fetching 2 files: 100%|██████████████████████████████████████████████████| 2/2 [00:
00<00:00, 24672.38it/s]
Loading weights: 100%|██████████████████████████████████████████████████| 824/824 [0
0:00<00:00, 861.22it/s]
Model and processor loaded successfully.
```

	image	response	true	time-error (in frames)	dimension error (log of ratio)	reasoning
0	No description has been provided for this image	102, 10	83, 42	19	-1.435085	\n102, 10
1	No description has been provided for this image	198, 11	166, 16	32	-0.374693	\n198, 11
2	No description has been provided for this image	249, 10	249, 260	0	-3.258097	\n249, 10
3	No description has been provided for this image	342, 11	332, 469	10	-3.752707	\n342, 11
4	No description has been provided for this image	222, 10	415, 418	-193	-3.732896	\n222, 10
5	No description has been provided for this image	228, 10	498, 10	-270	0.000000	\n228, 10

Ans: Few-shot is much better. We now have a good idea of the spinup time frames (though still off on exact times after spinup). The dimensionality is still completely wrong (surprising that it's still too hard a question).

Added structure didn't seem to help too much; in fact, reasoning required a variable token length, so the fixed (numeric) output structure conflicted with the reasoning sequence (so reasoning was removed in the final prompt). Also, the few-shot model required a random shuffle of inputs, to avoid coarse pattern-matching that ignored the image entirely (it just output a constant rise in the simulation time).

Problem 5: LoRA Fine-Tuning (20 points)

In this step, you'll fine-tune a **Vision-Language Model (VLM)** using **LoRA (Low-Rank Adaptation)** on your dataset.
This exercise will help you understand how different hyperparameters influence performance, GPU memory usage, and output quality.

Instructions

Run the code block below.
If you followed the `mmai-data` example, the script should automatically detect and load your training dataset.

Adjust and experiment with

- **Number of epochs** (`NUM_EPOCHS`)
- **Learning rate** (`LR`)
- **Batch size per device** (`BSZ_PER_DEV`)
- **Gradient accumulation steps** (`GRAD_ACCUM`)
- **Evaluation split ratio** (`EVAL_SPLIT`)
- **Random seed** (`SEED`)
- **Sequence length** (`MAX_SEQ_LEN`)
- **Image resolution** (`SHORTEST_EDGE`)

- **LoRA rank** (`LORA_R`)
- **LoRA alpha** (`LORA_ALPHA`)
- **LoRA dropout** (`LORA_DROPOUT`)
- **LoRA target modules** (`LORA_TARGET`)

```
# ============================================================
# ######################## CHANGE ME #########################
# ============================================================

# (Modify the parameters below in the Colab cell)

# ============================================================
# ###################### END CHANGE ME #######################
# ============================================================
```

Q&A

Q: What should I do if I encounter an out-of-memory issue?
A: Your image might be too large. Try resizing it by adding the following line back into your code and experiment with different pixel values:

```
img.thumbnail((128, 128))  # NOTE: If you run into an out-of-memory error, try adding this line back.
```

In [190…

```
!uv pip install peft
```

```
Using Python 3.12.12 environment at: /orcd/home/002/a1744874/ml/courses/.venv
Resolved 55 packages in 2.19s
Prepared 1 package in 1.02s
Installed 1 package in 266ms
 + peft==0.18.1
```

In [42]:

```
# ==== Qwen2.5-VL-3B-Instruct • FP16 LoRA ====

from IPython.display import display, HTML
import os, io, json, requests, torch, random, hashlib
from dataclasses import dataclass
from typing import Any, Dict, List
from PIL import Image
from torch.utils.data import Dataset
import torch.nn as nn
from transformers import (
    AutoProcessor,
    Qwen2_5_VLForConditionalGeneration,
    Trainer,
    TrainingArguments,
)
from peft import LoraConfig, get_peft_model

# Environment hygiene
os.environ["TOKENIZERS_PARALLELISM"] = "false"
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True,max_split_size_mb:64"

# ============================================================
# ######################## CHANGE ME #########################
# ============================================================
# Training hyperparameters
```

```python
NUM_EPOCHS: int  = 1
LR: float        = 1e-4
BSZ_PER_DEV: int = 8
GRAD_ACCUM: int  = 1
EVAL_SPLIT: float = 0.1
SEED: int        = 42

# Collator / sequence shaping
MAX_SEQ_LEN: int = 512   # try 384 if VRAM is tight

# Image preprocessing
SHORTEST_EDGE: int = 512 #288  # smaller saves VRAM
# ============================================================
# ###################### END CHANGE ME #######################
# ============================================================


# SYSTEM CONFIG
# Paths
DATA_JSONL: str  = "mmai-data/data.jsonl"
OUTPUT_DIR: str  = "qwen2_5_vl_lora_fp16_t4"

MODEL_ID: str     = "Qwen/Qwen2.5-VL-3B-Instruct"
CACHE_DIR: str    = "cache_images"
IMAGE_TIMEOUT: int = 15

# LoRA configuration (attention-only keeps memory low)
LORA_R: int          = 16 #4
LORA_ALPHA: int      = 8
LORA_DROPOUT: float  = 0.005
LORA_TARGET: list[str] = ["q_proj", "k_proj", "v_proj", "o_proj"]

# Device / dtype policy
FORCE_CPU: bool   = False
DTYPE_IF_GPU      = torch.float16
DTYPE_IF_CPU      = torch.float32
```

In [43]:

```python
# Repro and cache dirs
torch.manual_seed(SEED); random.seed(SEED)
os.makedirs(CACHE_DIR, exist_ok=True)
os.makedirs(OUTPUT_DIR, exist_ok=True)


# --------------------
# Demo data (create if missing)
# --------------------
def _ensure_sample_data(path: str):
    # Ensure parent directory exists
    os.makedirs(os.path.dirname(path), exist_ok=True)

    if os.path.exists(path):
      display(HTML(
          "<div style='color:white; background-color:#2e7d32; padding:10px; border-radius:6px;'>"
          "<strong>Using custom training data:</strong> "
          f"Loaded dataset from <code>{path}</code>. "
          "Proceeding with user-provided images and JSONL file."
          "</div>"
      ))
      return

    demo = [
```

```
        {
            "image": "http://images.cocodataset.org/val2017/000000039769.jpg",
            "question": "List objects you see.",
            "answer": "cat, sofa, blanket, remote, cushion, tail, paw"
        },
        {
            "image": "http://images.cocodataset.org/val2017/000000001532.jpg",
            "question": "List objects you see.",
            "answer": "car, truck, road, bridge, exit sign, lamppost, building, sky"
        },
    ]
    with open(path, "w") as f:
        for r in demo: f.write(json.dumps(r) + "\n")

    # Print a red warning box (works in Colab/Jupyter)
    display(HTML(
        "<div style='color:white; background-color:#b71c1c; padding:10px; border-radius:6px;'>"
        "<strong>Warning:</strong> No dataset found — using built-in <code>sample data</code> (2
        "Please replace with your own dataset of at least 20 images for training."
        "</div>"
    ))

_ensure_sample_data(DATA_JSONL)


# --------------------
# Minimal JSONL dataset
# --------------------
class JsonlVisionLangDataset(Dataset):
    def __init__(self, jsonl_path: str):
        self.samples: list[dict] = []
        with open(jsonl_path, "r") as f:
            for line in f:
                line = line.strip()
                if not line: continue
                ex = json.loads(line)
                if {"image","question","answer"} - set(ex.keys()): continue
                self.samples.append(ex)
    def __len__(self): return len(self.samples)
    def __getitem__(self, idx: int) -> Dict[str, Any]: return self.samples[idx]

full_ds = JsonlVisionLangDataset(DATA_JSONL)

# Manual split
n = len(full_ds); n_val = max(1, int(n * EVAL_SPLIT))
idx = list(range(n)); random.shuffle(idx)
val_idx = set(idx[:n_val])
train_data = [full_ds[i] for i in range(n) if i not in val_idx]
val_data   = [full_ds[i] for i in range(n) if i in val_idx]

class ListDataset(Dataset):
    def __init__(self, data_list): self.data_list = data_list
    def __len__(self): return len(self.data_list)
    def __getitem__(self, i): return self.data_list[i]


# --------------------
# Cache images locally (avoid network hiccups)
# --------------------
BASE_DIR = os.path.dirname(DATA_JSONL)

def cache_image(url_or_path: str) -> str:
```

```
    # Remote URL: download and cache
    if url_or_path.startswith(("http://", "https://")):
        h = hashlib.md5(url_or_path.encode()).hexdigest()
        local = os.path.join(CACHE_DIR, f"{h}.jpg")
        if not os.path.exists(local):
            r = requests.get(url_or_path, timeout=IMAGE_TIMEOUT); r.raise_for_status()
            with open(local, "wb") as f: f.write(r.content)
        return local

    # Local path: make absolute relative to the JSONL file
    candidate = url_or_path
    if not os.path.isabs(candidate):
        candidate = os.path.join(BASE_DIR, url_or_path)

    if not os.path.exists(candidate):
        raise FileNotFoundError(
            f"Image not found: {candidate} (from '{url_or_path}'). "
            f"Expected under {BASE_DIR}/"
        )
    return candidate


for ex in train_data: ex["image"] = cache_image(ex["image"])
for ex in val_data:   ex["image"] = cache_image(ex["image"])

train_ds = ListDataset(train_data)
val_ds   = ListDataset(val_data)


# --------------------
# Image loader
# --------------------
def load_image(img_path: str) -> Image.Image:
    img = Image.open(img_path).convert("RGB")
    # img.thumbnail((128, 128))  # NOTE: if you run into out of memory error, try adding this li
    return img


# --------------------
# Processor + Model (FP16 on GPU, FP32 on CPU)
# --------------------
use_cuda = torch.cuda.is_available() and not FORCE_CPU
torch_dtype = DTYPE_IF_GPU if use_cuda else DTYPE_IF_CPU
device_map = "auto" if use_cuda else None

processor = AutoProcessor.from_pretrained(MODEL_ID, trust_remote_code=True)
model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
    MODEL_ID,
    dtype=torch_dtype,            # transformers v5 uses 'dtype'
    device_map=device_map,
    low_cpu_mem_usage=True,
    trust_remote_code=True,
)
```

Using custom training data: Loaded dataset from `mmai-data/data.jsonl`. Proceeding with user-provided images and JSONL file.

```
Fetching 2 files: 100%|██████████████████████████████████████████████████| 2/2 [00:
00<00:00, 13662.23it/s]
Loading weights: 100%|═══════════════════════════════════════════════════| 824/824 [0
0:03<00:00, 263.29it/s]
```

In [ ]:

```python
# Smaller images to save VRAM
try:
    if hasattr(processor, "image_processor") and hasattr(processor.image_processor, "size"):
        processor.image_processor.size = {
            "shortest_edge": int(SHORTEST_EDGE),
            "longest_edge": int(SHORTEST_EDGE * 4),
        }
        print(f"Set image shortest_edge to {SHORTEST_EDGE}, longest_edge to {SHORTEST_EDGE * 4}"
except Exception as e:
    print("Skip image size tweak:", e)

# Enable gradient checkpointing; avoid k-bit prep (saves VRAM)
model.gradient_checkpointing_enable()
model.enable_input_require_grads()
model.config.use_cache = False


# --------------------
# LoRA (attention-only)
# --------------------
lora_cfg = LoraConfig(
    r=LORA_R,
    lora_alpha=LORA_ALPHA,
    target_modules=LORA_TARGET,
    lora_dropout=LORA_DROPOUT,
    bias="none",
    task_type="CAUSAL_LM",
)
model = get_peft_model(model, lora_cfg)
model.print_trainable_parameters()


# --------------------
# Collator (truncate to keep sequences small)
# --------------------
@dataclass
class VLDataCollator:
    processor: Any
    def __call__(self, features: List[Dict[str, Any]]) -> Dict[str, torch.Tensor]:
        images, texts = [], []
        for ex in features:
            img = load_image(ex["image"])
            messages = [
                {"role": "user", "content": [
                    {"type":"image","image": img},
                    {"type":"text","text": ex["question"]},
                ]},
                {"role": "assistant", "content": [
                    {"type":"text","text": ex["answer"]},
                ]},
            ]
            text = self.processor.apply_chat_template(messages, tokenize=False, add_generation_pr
            images.append(img); texts.append(text)

        batch = self.processor(
            text=texts,
```

```
            images=images,
            padding=True,
            truncation=True,
            max_length=MAX_SEQ_LEN,
            return_tensors="pt",
        )
        labels = batch["input_ids"].clone()
        labels[batch["attention_mask"] == 0] = -100
        batch["labels"] = labels

        for im in images:
            try: im.close()
            except: pass

        return batch

collator = VLDataCollator(processor)


# --------------------
# FP16 loss trainer to avoid fp32 upcast OOM
# --------------------
class FP16CLMTrainer(Trainer):
    def compute_loss(
        self,
        model,
        inputs,
        return_outputs=False,
        num_items_in_batch=None,   # v5 may pass this
        **kwargs,
    ):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits  # keep fp16 path if available

        # Shift for causal LM
        shift_logits = logits[..., :-1, :].contiguous()
        shift_labels = labels[..., 1:].contiguous()

        loss_fct = nn.CrossEntropyLoss(ignore_index=-100)
        loss = loss_fct(
            shift_logits.view(-1, shift_logits.size(-1)),
            shift_labels.view(-1),
        )
        return (loss, outputs) if return_outputs else loss


# --------------------
# TrainingArguments (Transformers v5+ naming)
# --------------------
args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    num_train_epochs=NUM_EPOCHS,
    max_steps=2000,
    per_device_train_batch_size=BSZ_PER_DEV,
    per_device_eval_batch_size=1,
    dataloader_num_workers=0,
    gradient_accumulation_steps=GRAD_ACCUM,
    learning_rate=LR,
    warmup_ratio=0.03,
    lr_scheduler_type="cosine",
    logging_steps=1,
```

```
    eval_strategy="no",              # keep simple; eval loop optional
    save_strategy="steps",
    save_steps=10_000,

    fp16=False, bf16=use_cuda,       # FP16 only if GPU
    gradient_checkpointing=True,
    optim="adamw_torch",
    report_to=[],
    remove_unused_columns=False,
)

trainer = FP16CLMTrainer(
    model=model,
    args=args,
    data_collator=collator,
    train_dataset=train_ds,
    eval_dataset=val_ds,
)
```

In [48]:

```
trainer.train()

# Save LoRA adapters + processor
trainer.model.save_pretrained(OUTPUT_DIR)
processor.save_pretrained(OUTPUT_DIR)
print("Training complete. LoRA adapters saved to:", OUTPUT_DIR)
```

[2000/2000 13:20, Epoch 1/2]

Step	Training Loss
1	0.318625
2	0.321620
3	0.334054
4	0.310619
5	0.320059
6	0.323536
7	0.324883
8	0.312495
9	0.322927
10	0.333222
11	0.323585
12	0.337450
13	0.323812
14	0.325302
15	0.332025
16	0.334366
17	0.326387
18	0.331737
19	0.334471
20	0.328756
21	0.318721
22	0.320759
23	0.317284
24	0.320926
25	0.321865
26	0.331033
27	0.330104
28	0.318486
29	0.324344
30	0.340262
31	0.319978
32	0.327732
33	0.319977

Step	Training Loss
34	0.321424
35	0.310661
36	0.324221
37	0.330953
38	0.325563
39	0.327832
40	0.327314
41	0.325789
42	0.328078
43	0.326645
44	0.318972
45	0.325818
46	0.337701
47	0.328849
48	0.330390
49	0.332293
50	0.332001
51	0.329206
52	0.327205
53	0.339988
54	0.321593
55	0.323370
56	0.305242
57	0.325400
58	0.318643
59	0.333037
60	0.327572
61	0.334812
62	0.320213
63	0.340250
64	0.335569
65	0.325843
66	0.318142
67	0.330830

Step	Training Loss
68	0.333351
69	0.325145
70	0.320288
71	0.321264
72	0.354342
73	0.321751
74	0.326392
75	0.326948
76	0.316775
77	0.322483
78	0.336137
79	0.321131
80	0.325142
81	0.335543
82	0.320416
83	0.333512
84	0.327525
85	0.341494
86	0.319414
87	0.316681
88	0.329692
89	0.338788
90	0.327560
91	0.321373
92	0.323448
93	0.331695
94	0.337725
95	0.329898
96	0.325572
97	0.331051
98	0.330470
99	0.334342
100	0.332037
101	0.335587

Step	Training Loss
102	0.329703
103	0.324719
104	0.320836
105	0.336697
106	0.334098
107	0.327827
108	0.325956
109	0.329669
110	0.320492
111	0.329347
112	0.330918
113	0.332760
114	0.334274
115	0.331480
116	0.335866
117	0.326990
118	0.336991
119	0.326670
120	0.325062
121	0.316908
122	0.316935
123	0.328828
124	0.329533
125	0.319712
126	0.327157
127	0.331908
128	0.325983
129	0.330793
130	0.321614
131	0.331485
132	0.329779
133	0.324158
134	0.316256
135	0.331048

Step	Training Loss
136	0.337816
137	0.318484
138	0.337030
139	0.322736
140	0.315721
141	0.320255
142	0.338323
143	0.333398
144	0.329534
145	0.326710
146	0.328470
147	0.332858
148	0.321594
149	0.330726
150	0.338237
151	0.333804
152	0.318002
153	0.322604
154	0.324848
155	0.338022
156	0.324823
157	0.318251
158	0.322972
159	0.332255
160	0.322860
161	0.322936
162	0.318986
163	0.339631
164	0.326079
165	0.325120
166	0.329786
167	0.323479
168	0.326861
169	0.316229

Step	Training Loss
170	0.315008
171	0.314271
172	0.330179
173	0.324830
174	0.321849
175	0.324614
176	0.326552
177	0.323034
178	0.342576
179	0.334774
180	0.337564
181	0.334841
182	0.326268
183	0.315559
184	0.333889
185	0.328053
186	0.328641
187	0.333597
188	0.322564
189	0.326025
190	0.319029
191	0.331906
192	0.327016
193	0.337935
194	0.319169
195	0.320712
196	0.327810
197	0.340251
198	0.331075
199	0.332664
200	0.321886
201	0.320819
202	0.328292
203	0.316243

Step	Training Loss
204	0.326790
205	0.336568
206	0.333285
207	0.329069
208	0.312573
209	0.338361
210	0.333266
211	0.328391
212	0.323079
213	0.330600
214	0.321147
215	0.330436
216	0.324197
217	0.335282
218	0.334498
219	0.314036
220	0.326455
221	0.328670
222	0.321977
223	0.332291
224	0.332171
225	0.335073
226	0.322138
227	0.342059
228	0.333734
229	0.338387
230	0.331732
231	0.325003
232	0.322226
233	0.317441
234	0.327816
235	0.338803
236	0.325936
237	0.319069

Step	Training Loss
238	0.325425
239	0.322338
240	0.325398
241	0.318747
242	0.317380
243	0.327065
244	0.333474
245	0.322716
246	0.327746
247	0.326493
248	0.318373
249	0.338610
250	0.324266
251	0.315252
252	0.343885
253	0.332056
254	0.325959
255	0.325468
256	0.321617
257	0.328372
258	0.323758
259	0.334146
260	0.311591
261	0.320457
262	0.327753
263	0.330309
264	0.336055
265	0.338268
266	0.314766
267	0.320091
268	0.326767
269	0.326573
270	0.309013
271	0.317625

Step	Training Loss
272	0.333799
273	0.328231
274	0.318989
275	0.336497
276	0.345899
277	0.321108
278	0.322600
279	0.313463
280	0.321314
281	0.323813
282	0.319088
283	0.325073
284	0.321666
285	0.317566
286	0.312449
287	0.323555
288	0.316290
289	0.326570
290	0.319489
291	0.333498
292	0.329009
293	0.323739
294	0.341648
295	0.334710
296	0.339647
297	0.316810
298	0.324827
299	0.330504
300	0.320542
301	0.326524
302	0.325719
303	0.324037
304	0.326193
305	0.336625

Step	Training Loss
306	0.329026
307	0.324003
308	0.314955
309	0.330580
310	0.336775
311	0.320316
312	0.319418
313	0.331997
314	0.328631
315	0.319153
316	0.318262
317	0.319635
318	0.318064
319	0.321135
320	0.336416
321	0.324047
322	0.316537
323	0.333863
324	0.312087
325	0.327382
326	0.326541
327	0.326817
328	0.343141
329	0.327954
330	0.331265
331	0.330662
332	0.322604
333	0.321067
334	0.309185
335	0.317931
336	0.316241
337	0.322339
338	0.331287
339	0.342874

Step	Training Loss
340	0.326176
341	0.326376
342	0.320113
343	0.312225
344	0.324639
345	0.313807
346	0.332064
347	0.319145
348	0.326510
349	0.323842
350	0.336189
351	0.319742
352	0.330761
353	0.329639
354	0.331918
355	0.330272
356	0.327471
357	0.328286
358	0.328250
359	0.317515
360	0.331324
361	0.306429
362	0.337337
363	0.322515
364	0.315325
365	0.334366
366	0.328378
367	0.325168
368	0.327464
369	0.334463
370	0.333071
371	0.314672
372	0.316484
373	0.335700

Step	Training Loss
374	0.318590
375	0.329472
376	0.336555
377	0.327350
378	0.316324
379	0.316392
380	0.322947
381	0.319684
382	0.320498
383	0.316760
384	0.322771
385	0.332188
386	0.335347
387	0.327522
388	0.328217
389	0.330859
390	0.316061
391	0.328399
392	0.331133
393	0.324825
394	0.326673
395	0.318301
396	0.316627
397	0.327208
398	0.332758
399	0.322716
400	0.315160
401	0.319154
402	0.333067
403	0.333442
404	0.331365
405	0.319502
406	0.333577
407	0.345124

Step	Training Loss
408	0.326193
409	0.326584
410	0.339269
411	0.322038
412	0.327309
413	0.315917
414	0.318182
415	0.319454
416	0.324214
417	0.332935
418	0.323614
419	0.324416
420	0.324164
421	0.329796
422	0.326991
423	0.334553
424	0.317274
425	0.327496
426	0.334017
427	0.328419
428	0.324596
429	0.330444
430	0.322762
431	0.314534
432	0.338066
433	0.322856
434	0.306875
435	0.315168
436	0.323570
437	0.328455
438	0.330344
439	0.320055
440	0.321324
441	0.324460

Step	Training Loss
442	0.318863
443	0.333652
444	0.321098
445	0.321801
446	0.327215
447	0.329513
448	0.327068
449	0.326785
450	0.321802
451	0.314802
452	0.345395
453	0.331261
454	0.327713
455	0.317204
456	0.320418
457	0.332253
458	0.315671
459	0.329514
460	0.332790
461	0.318501
462	0.337303
463	0.334067
464	0.327483
465	0.329362
466	0.343637
467	0.318158
468	0.318406
469	0.322324
470	0.321590
471	0.329951
472	0.326898
473	0.327553
474	0.330589
475	0.325293

Step	Training Loss
476	0.317028
477	0.342278
478	0.321497
479	0.328931
480	0.315116
481	0.310718
482	0.322554
483	0.324572
484	0.317186
485	0.333063
486	0.329012
487	0.313164
488	0.325814
489	0.317122
490	0.318001
491	0.323534
492	0.310181
493	0.331329
494	0.319242
495	0.320542
496	0.311301
497	0.321522
498	0.326532
499	0.341742
500	0.317118
501	0.320082
502	0.312469
503	0.312138
504	0.331340
505	0.319864
506	0.332043
507	0.307191
508	0.323883
509	0.325131

Step	Training Loss
510	0.333224
511	0.307112
512	0.319598
513	0.323169
514	0.326412
515	0.330247
516	0.319057
517	0.319298
518	0.345209
519	0.320646
520	0.317279
521	0.331641
522	0.326670
523	0.311311
524	0.325544
525	0.329638
526	0.315007
527	0.310019
528	0.307478
529	0.321381
530	0.329356
531	0.309576
532	0.316508
533	0.328131
534	0.322435
535	0.332522
536	0.323281
537	0.319058
538	0.318388
539	0.318882
540	0.320221
541	0.329220
542	0.317605
543	0.324207

Step	Training Loss
544	0.340994
545	0.329287
546	0.325825
547	0.319148
548	0.320585
549	0.332817
550	0.330172
551	0.316577
552	0.327121
553	0.323514
554	0.333992
555	0.321358
556	0.326694
557	0.324362
558	0.318905
559	0.327435
560	0.325853
561	0.324838
562	0.324970
563	0.317443
564	0.323392
565	0.336998
566	0.317297
567	0.315866
568	0.325571
569	0.334855
570	0.330620
571	0.311349
572	0.331795
573	0.318086
574	0.318737
575	0.319009
576	0.327305
577	0.324717

Step	Training Loss
578	0.309681
579	0.341775
580	0.323883
581	0.319648
582	0.316382
583	0.321450
584	0.316089
585	0.329070
586	0.318788
587	0.317027
588	0.332302
589	0.315969
590	0.333334
591	0.333008
592	0.327087
593	0.328772
594	0.316230
595	0.328841
596	0.333878
597	0.326831
598	0.336247
599	0.323509
600	0.312771
601	0.326278
602	0.319038
603	0.333875
604	0.313036
605	0.329587
606	0.329952
607	0.327293
608	0.313773
609	0.332540
610	0.326728
611	0.312551

Step	Training Loss
612	0.332111
613	0.314727
614	0.329046
615	0.336323
616	0.329011
617	0.316981
618	0.314687
619	0.330285
620	0.326730
621	0.320685
622	0.322302
623	0.335043
624	0.329436
625	0.320371
626	0.328335
627	0.312953
628	0.325186
629	0.330974
630	0.325478
631	0.310457
632	0.321656
633	0.315319
634	0.333443
635	0.322106
636	0.328283
637	0.313894
638	0.316636
639	0.322681
640	0.325593
641	0.317631
642	0.315683
643	0.327753
644	0.319878
645	0.328962

Step	Training Loss
646	0.325956
647	0.314584
648	0.336441
649	0.323610
650	0.322909
651	0.330645
652	0.333293
653	0.316354
654	0.317079
655	0.325142
656	0.318307
657	0.324949
658	0.322250
659	0.315892
660	0.330701
661	0.324105
662	0.324325
663	0.313266
664	0.334961
665	0.329076
666	0.330670
667	0.318701
668	0.322574
669	0.317227
670	0.319085
671	0.339737
672	0.328454
673	0.319011
674	0.335497
675	0.320035
676	0.331659
677	0.322245
678	0.315579
679	0.330981

Step	Training Loss
680	0.320392
681	0.315591
682	0.326575
683	0.332859
684	0.314659
685	0.331659
686	0.317547
687	0.323998
688	0.334672
689	0.315246
690	0.329270
691	0.320971
692	0.315755
693	0.321684
694	0.335635
695	0.331481
696	0.327282
697	0.315694
698	0.318411
699	0.314952
700	0.308057
701	0.317380
702	0.316177
703	0.328346
704	0.317772
705	0.332877
706	0.330928
707	0.313398
708	0.332654
709	0.326826
710	0.316886
711	0.330927
712	0.315202
713	0.322019

Step	Training Loss
714	0.320870
715	0.328986
716	0.323238
717	0.332954
718	0.321582
719	0.328229
720	0.325364
721	0.324328
722	0.324741
723	0.335488
724	0.328416
725	0.331859
726	0.336466
727	0.336652
728	0.316328
729	0.321295
730	0.314290
731	0.339194
732	0.331300
733	0.332188
734	0.328745
735	0.338294
736	0.324018
737	0.327493
738	0.317641
739	0.314397
740	0.315774
741	0.307249
742	0.319699
743	0.326881
744	0.312856
745	0.326887
746	0.350271
747	0.324685

Step	Training Loss
748	0.339602
749	0.334031
750	0.336140
751	0.322674
752	0.320042
753	0.332986
754	0.338556
755	0.329351
756	0.312449
757	0.312795
758	0.328955
759	0.327745
760	0.320342
761	0.326476
762	0.317815
763	0.308995
764	0.324721
765	0.330488
766	0.318685
767	0.309521
768	0.323572
769	0.316075
770	0.335111
771	0.331043
772	0.315875
773	0.328113
774	0.320902
775	0.317311
776	0.320577
777	0.313233
778	0.335649
779	0.335168
780	0.309374
781	0.325142

Step	Training Loss
782	0.318347
783	0.319479
784	0.308928
785	0.334975
786	0.323044
787	0.321810
788	0.327907
789	0.309500
790	0.328165
791	0.323894
792	0.319959
793	0.325336
794	0.318132
795	0.330827
796	0.327513
797	0.325041
798	0.326907
799	0.316928
800	0.319108
801	0.319275
802	0.318384
803	0.343804
804	0.326356
805	0.324909
806	0.322477
807	0.323138
808	0.316375
809	0.321716
810	0.319596
811	0.324577
812	0.320772
813	0.324661
814	0.312082
815	0.321333

Step	Training Loss
816	0.310021
817	0.335823
818	0.303553
819	0.331587
820	0.323207
821	0.334921
822	0.305975
823	0.298401
824	0.324499
825	0.316861
826	0.312486
827	0.312808
828	0.323990
829	0.319231
830	0.328605
831	0.320362
832	0.321954
833	0.314102
834	0.321711
835	0.311352
836	0.324667
837	0.317423
838	0.328162
839	0.326253
840	0.334115
841	0.324875
842	0.335728
843	0.327198
844	0.328481
845	0.321000
846	0.318934
847	0.321178
848	0.326689
849	0.311396

Step	Training Loss
850	0.317599
851	0.319764
852	0.311665
853	0.322436
854	0.331168
855	0.321920
856	0.312936
857	0.321233
858	0.331792
859	0.326741
860	0.321170
861	0.332159
862	0.326458
863	0.317682
864	0.328814
865	0.315789
866	0.325231
867	0.332889
868	0.332532
869	0.314753
870	0.311421
871	0.322444
872	0.316306
873	0.322670
874	0.331036
875	0.317842
876	0.321406
877	0.327462
878	0.328804
879	0.317497
880	0.331745
881	0.325420
882	0.322662
883	0.300440

Step	Training Loss
884	0.326137
885	0.328846
886	0.320966
887	0.329733
888	0.321022
889	0.327009
890	0.317262
891	0.325698
892	0.310738
893	0.322354
894	0.321046
895	0.322025
896	0.312690
897	0.310535
898	0.326084
899	0.337496
900	0.339749
901	0.325830
902	0.309498
903	0.325324
904	0.330837
905	0.318944
906	0.304227
907	0.319349
908	0.309667
909	0.330455
910	0.323793
911	0.328695
912	0.324102
913	0.328921
914	0.329527
915	0.312121
916	0.328253
917	0.318622

Step	Training Loss
918	0.317292
919	0.316743
920	0.311317
921	0.334952
922	0.334701
923	0.317001
924	0.324910
925	0.319121
926	0.320611
927	0.314799
928	0.319820
929	0.324381
930	0.316504
931	0.313463
932	0.330253
933	0.330443
934	0.327634
935	0.326468
936	0.323556
937	0.331696
938	0.331936
939	0.324310
940	0.327335
941	0.313334
942	0.321501
943	0.316054
944	0.313285
945	0.313505
946	0.326061
947	0.323397
948	0.325213
949	0.321930
950	0.315430
951	0.311436

Step	Training Loss
952	0.332141
953	0.323239
954	0.310357
955	0.319653
956	0.330329
957	0.327707
958	0.331385
959	0.311688
960	0.344777
961	0.324967
962	0.314816
963	0.325795
964	0.321455
965	0.310503
966	0.311563
967	0.323200
968	0.308973
969	0.317556
970	0.315678
971	0.308479
972	0.309961
973	0.309557
974	0.325770
975	0.322299
976	0.319452
977	0.333461
978	0.321547
979	0.325243
980	0.317761
981	0.335134
982	0.330077
983	0.313568
984	0.331696
985	0.322336

Step	Training Loss
986	0.325936
987	0.322623
988	0.327932
989	0.317589
990	0.309944
991	0.331268
992	0.322266
993	0.324441
994	0.310601
995	0.323007
996	0.306184
997	0.321421
998	0.313090
999	0.326472
1000	0.317958
1001	0.326360
1002	0.330449
1003	0.323641
1004	0.321323
1005	0.332197
1006	0.314700
1007	0.327445
1008	0.318399
1009	0.321322
1010	0.312328
1011	0.328293
1012	0.314841
1013	0.313818
1014	0.328959
1015	0.344025
1016	0.325165
1017	0.319437
1018	0.302712
1019	0.310347

Step	Training Loss
1020	0.330839
1021	0.320915
1022	0.323576
1023	0.323917
1024	0.309630
1025	0.324341
1026	0.319547
1027	0.326095
1028	0.323806
1029	0.331354
1030	0.310637
1031	0.312801
1032	0.318877
1033	0.320916
1034	0.329318
1035	0.337360
1036	0.315367
1037	0.328719
1038	0.339785
1039	0.332320
1040	0.304767
1041	0.314017
1042	0.321792
1043	0.315229
1044	0.316889
1045	0.328917
1046	0.317534
1047	0.326514
1048	0.318029
1049	0.328023
1050	0.310761
1051	0.312299
1052	0.314984
1053	0.329594

Step	Training Loss
1054	0.317602
1055	0.305563
1056	0.312723
1057	0.327162
1058	0.318451
1059	0.331547
1060	0.320633
1061	0.319867
1062	0.331501
1063	0.321138
1064	0.326789
1065	0.313375
1066	0.324934
1067	0.312988
1068	0.334685
1069	0.314969
1070	0.327566
1071	0.320202
1072	0.325500
1073	0.303169
1074	0.319225
1075	0.316004
1076	0.319261
1077	0.336327
1078	0.337038
1079	0.325476
1080	0.322821
1081	0.333614
1082	0.312294
1083	0.330560
1084	0.312339
1085	0.328130
1086	0.322010
1087	0.322280

Step	Training Loss
1088	0.309327
1089	0.327204
1090	0.309171
1091	0.336474
1092	0.313071
1093	0.322316
1094	0.336450
1095	0.326521
1096	0.309372
1097	0.324188
1098	0.322072
1099	0.321471
1100	0.330050
1101	0.327772
1102	0.317160
1103	0.321595
1104	0.324076
1105	0.326412
1106	0.320510
1107	0.323833
1108	0.309734
1109	0.328389
1110	0.316999
1111	0.325309
1112	0.326685
1113	0.316439
1114	0.314087
1115	0.322004
1116	0.324475
1117	0.331639
1118	0.316322
1119	0.335792
1120	0.331379
1121	0.307945

Step	Training Loss
1122	0.333052
1123	0.317713
1124	0.320477
1125	0.312508
1126	0.326521
1127	0.321122
1128	0.314996
1129	0.326950
1130	0.300596
1131	0.312766
1132	0.323236
1133	0.329865
1134	0.316048
1135	0.315328
1136	0.320537
1137	0.315345
1138	0.326705
1139	0.321811
1140	0.323360
1141	0.328067
1142	0.325812
1143	0.302881
1144	0.313640
1145	0.321789
1146	0.329259
1147	0.323599
1148	0.325637
1149	0.312194
1150	0.318546
1151	0.318033
1152	0.310225
1153	0.321167
1154	0.305871
1155	0.305200

Step	Training Loss
1156	0.306588
1157	0.308845
1158	0.312049
1159	0.327159
1160	0.308594
1161	0.330839
1162	0.325840
1163	0.301840
1164	0.328434
1165	0.321093
1166	0.318073
1167	0.325330
1168	0.327354
1169	0.319251
1170	0.315021
1171	0.325291
1172	0.316201
1173	0.302926
1174	0.312769
1175	0.329920
1176	0.319930
1177	0.317437
1178	0.327688
1179	0.333923
1180	0.318618
1181	0.309640
1182	0.309475
1183	0.315203
1184	0.321443
1185	0.319466
1186	0.319843
1187	0.314430
1188	0.311363
1189	0.312432

Step	Training Loss
1190	0.318745
1191	0.319026
1192	0.324329
1193	0.313017
1194	0.325009
1195	0.314466
1196	0.303843
1197	0.311710
1198	0.321242
1199	0.332371
1200	0.319908
1201	0.325158
1202	0.323589
1203	0.305887
1204	0.331923
1205	0.328619
1206	0.321667
1207	0.311434
1208	0.320540
1209	0.325900
1210	0.324486
1211	0.324463
1212	0.305087
1213	0.332763
1214	0.316637
1215	0.324398
1216	0.322031
1217	0.308276
1218	0.334788
1219	0.323774
1220	0.318016
1221	0.313183
1222	0.327687
1223	0.323821

Step	Training Loss
1224	0.321663
1225	0.315328
1226	0.338290
1227	0.331084
1228	0.323456
1229	0.320612
1230	0.311365
1231	0.316832
1232	0.319441
1233	0.309309
1234	0.310290
1235	0.317783
1236	0.316862
1237	0.326743
1238	0.308647
1239	0.319920
1240	0.309740
1241	0.317366
1242	0.330055
1243	0.335524
1244	0.328417
1245	0.314295
1246	0.317437
1247	0.324176
1248	0.319605
1249	0.317172
1250	0.306802
1251	0.329996
1252	0.316924
1253	0.323070
1254	0.333147
1255	0.319374
1256	0.335790
1257	0.318898

Step	Training Loss
1258	0.327912
1259	0.318383
1260	0.321108
1261	0.307177
1262	0.332400
1263	0.315900
1264	0.307990
1265	0.316304
1266	0.321982
1267	0.321580
1268	0.311070
1269	0.325002
1270	0.328458
1271	0.318679
1272	0.310383
1273	0.321862
1274	0.329758
1275	0.327864
1276	0.313508
1277	0.323561
1278	0.323655
1279	0.316484
1280	0.334489
1281	0.320560
1282	0.309853
1283	0.319587
1284	0.325794
1285	0.325086
1286	0.324800
1287	0.314611
1288	0.306033
1289	0.319550
1290	0.320824
1291	0.313713

Step	Training Loss
1292	0.327452
1293	0.308712
1294	0.311402
1295	0.326092
1296	0.331406
1297	0.320774
1298	0.314429
1299	0.315913
1300	0.317684
1301	0.310771
1302	0.314725
1303	0.334584
1304	0.316529
1305	0.322987
1306	0.316966
1307	0.326528
1308	0.328838
1309	0.319191
1310	0.326766
1311	0.317251
1312	0.318007
1313	0.335716
1314	0.311847
1315	0.328793
1316	0.320489
1317	0.337322
1318	0.311859
1319	0.325860
1320	0.302787
1321	0.336462
1322	0.321305
1323	0.322524
1324	0.304206
1325	0.320680

Step	Training Loss
1326	0.334233
1327	0.325631
1328	0.313664
1329	0.320685
1330	0.317674
1331	0.312495
1332	0.325058
1333	0.318219
1334	0.315592
1335	0.327474
1336	0.312815
1337	0.316678
1338	0.312379
1339	0.299566
1340	0.316294
1341	0.329213
1342	0.319673
1343	0.327310
1344	0.311959
1345	0.301873
1346	0.327572
1347	0.313208
1348	0.327212
1349	0.312334
1350	0.315172
1351	0.332110
1352	0.331293
1353	0.310091
1354	0.322866
1355	0.328968
1356	0.287640
1357	0.313297
1358	0.292199
1359	0.306767

Step	Training Loss
1360	0.332447
1361	0.324885
1362	0.324495
1363	0.311154
1364	0.319647
1365	0.321502
1366	0.318374
1367	0.319797
1368	0.319300
1369	0.331223
1370	0.317298
1371	0.333538
1372	0.321152
1373	0.324479
1374	0.319367
1375	0.307998
1376	0.311855
1377	0.318690
1378	0.318117
1379	0.322404
1380	0.314224
1381	0.321252
1382	0.309509
1383	0.333926
1384	0.308004
1385	0.305369
1386	0.316530
1387	0.318490
1388	0.333361
1389	0.319190
1390	0.307949
1391	0.296637
1392	0.329462
1393	0.322387

Step	Training Loss
1394	0.319716
1395	0.323108
1396	0.321331
1397	0.322139
1398	0.321888
1399	0.327243
1400	0.322851
1401	0.314252
1402	0.306024
1403	0.327260
1404	0.324629
1405	0.324527
1406	0.324017
1407	0.321487
1408	0.313316
1409	0.308077
1410	0.324795
1411	0.319616
1412	0.320905
1413	0.322582
1414	0.329630
1415	0.323203
1416	0.321265
1417	0.319969
1418	0.315796
1419	0.310892
1420	0.315487
1421	0.319985
1422	0.300096
1423	0.305340
1424	0.328835
1425	0.311926
1426	0.324921
1427	0.314188

Step	Training Loss
1428	0.288447
1429	0.312951
1430	0.315367
1431	0.316067
1432	0.307036
1433	0.320914
1434	0.319418
1435	0.315703
1436	0.310925
1437	0.313182
1438	0.325715
1439	0.328765
1440	0.320857
1441	0.302083
1442	0.323168
1443	0.318777
1444	0.317883
1445	0.336676
1446	0.306626
1447	0.322302
1448	0.321680
1449	0.320736
1450	0.310269
1451	0.324466
1452	0.330079
1453	0.319185
1454	0.322138
1455	0.324191
1456	0.323856
1457	0.320426
1458	0.314646
1459	0.321649
1460	0.316849
1461	0.314853

Step	Training Loss
1462	0.303534
1463	0.317623
1464	0.328538
1465	0.323135
1466	0.321912
1467	0.315891
1468	0.313094
1469	0.311551
1470	0.325340
1471	0.308672
1472	0.305527
1473	0.294115
1474	0.304872
1475	0.323549
1476	0.321963
1477	0.323897
1478	0.295792
1479	0.332345
1480	0.334544
1481	0.330685
1482	0.310170
1483	0.327699
1484	0.322632
1485	0.325082
1486	0.324532
1487	0.324479
1488	0.316492
1489	0.298076
1490	0.315705
1491	0.304531
1492	0.321075
1493	0.316457
1494	0.323657
1495	0.320178

Step	Training Loss
1496	0.325822
1497	0.324805
1498	0.315999
1499	0.308189
1500	0.324626
1501	0.312480
1502	0.332334
1503	0.328067
1504	0.308676
1505	0.327020
1506	0.333398
1507	0.308314
1508	0.308060
1509	0.329343
1510	0.318401
1511	0.311020
1512	0.307258
1513	0.317555
1514	0.328923
1515	0.323380
1516	0.328489
1517	0.320105
1518	0.313009
1519	0.325294
1520	0.328619
1521	0.318568
1522	0.333295
1523	0.317276
1524	0.306148
1525	0.327444
1526	0.318149
1527	0.330307
1528	0.315519
1529	0.333890

Step	Training Loss
1530	0.308793
1531	0.315948
1532	0.324570
1533	0.317049
1534	0.311137
1535	0.305224
1536	0.311774
1537	0.336761
1538	0.316621
1539	0.330593
1540	0.314953
1541	0.322469
1542	0.308496
1543	0.329739
1544	0.313557
1545	0.330655
1546	0.311269
1547	0.318397
1548	0.322770
1549	0.305840
1550	0.299259
1551	0.310013
1552	0.325113
1553	0.307547
1554	0.329516
1555	0.320709
1556	0.327943
1557	0.326602
1558	0.317030
1559	0.322199
1560	0.307598
1561	0.316768
1562	0.334723
1563	0.315046

Step	Training Loss
1564	0.311259
1565	0.298010
1566	0.322972
1567	0.318452
1568	0.313680
1569	0.320672
1570	0.321494
1571	0.328688
1572	0.328345
1573	0.313183
1574	0.316770
1575	0.307781
1576	0.322944
1577	0.316462
1578	0.330891
1579	0.313976
1580	0.337060
1581	0.319985
1582	0.303115
1583	0.313514
1584	0.338763
1585	0.299492
1586	0.330782
1587	0.315036
1588	0.310136
1589	0.329251
1590	0.324036
1591	0.326852
1592	0.308083
1593	0.321552
1594	0.309913
1595	0.328808
1596	0.320808
1597	0.304807

Step	Training Loss
1598	0.320635
1599	0.319606
1600	0.295088
1601	0.327856
1602	0.323010
1603	0.334498
1604	0.317976
1605	0.312850
1606	0.324862
1607	0.303851
1608	0.305561
1609	0.298476
1610	0.325214
1611	0.311949
1612	0.310167
1613	0.326878
1614	0.318708
1615	0.327232
1616	0.328483
1617	0.320334
1618	0.312290
1619	0.325143
1620	0.309226
1621	0.329559
1622	0.322970
1623	0.312188
1624	0.311425
1625	0.315478
1626	0.323761
1627	0.305722
1628	0.320813
1629	0.324144
1630	0.314747
1631	0.334591

Step	Training Loss
1632	0.323400
1633	0.326754
1634	0.326301
1635	0.323328
1636	0.330981
1637	0.311576
1638	0.331524
1639	0.320880
1640	0.315593
1641	0.320864
1642	0.317132
1643	0.303534
1644	0.324166
1645	0.330049
1646	0.320251
1647	0.324894
1648	0.298907
1649	0.329209
1650	0.322669
1651	0.327868
1652	0.314439
1653	0.310850
1654	0.316862
1655	0.314423
1656	0.322428
1657	0.304779
1658	0.308565
1659	0.314978
1660	0.317131
1661	0.320265
1662	0.319675
1663	0.312700
1664	0.323651
1665	0.316331

Step	Training Loss
1666	0.337327
1667	0.314917
1668	0.312797
1669	0.316822
1670	0.320171
1671	0.322856
1672	0.334507
1673	0.327835
1674	0.313485
1675	0.324808
1676	0.294548
1677	0.309813
1678	0.316994
1679	0.318677
1680	0.323790
1681	0.323165
1682	0.310826
1683	0.311700
1684	0.321016
1685	0.321123
1686	0.308449
1687	0.332577
1688	0.325202
1689	0.311343
1690	0.310535
1691	0.324308
1692	0.321126
1693	0.311298
1694	0.309180
1695	0.318365
1696	0.322145
1697	0.306044
1698	0.310266
1699	0.313010

Step	Training Loss
1700	0.323054
1701	0.331733
1702	0.320965
1703	0.314025
1704	0.329471
1705	0.318248
1706	0.317657
1707	0.318124
1708	0.312382
1709	0.327091
1710	0.314097
1711	0.313063
1712	0.325408
1713	0.322853
1714	0.312723
1715	0.319233
1716	0.323387
1717	0.306858
1718	0.309305
1719	0.330776
1720	0.325955
1721	0.321491
1722	0.312422
1723	0.315575
1724	0.323670
1725	0.321189
1726	0.315976
1727	0.319731
1728	0.316069
1729	0.322157
1730	0.312759
1731	0.307533
1732	0.309200
1733	0.320474

Step	Training Loss
1734	0.321909
1735	0.325925
1736	0.316520
1737	0.327151
1738	0.321610
1739	0.323992
1740	0.324594
1741	0.322131
1742	0.325549
1743	0.319088
1744	0.322171
1745	0.320393
1746	0.319536
1747	0.316580
1748	0.309860
1749	0.318470
1750	0.315108
1751	0.333657
1752	0.318929
1753	0.317600
1754	0.305995
1755	0.323599
1756	0.320419
1757	0.310582
1758	0.320451
1759	0.309540
1760	0.317422
1761	0.319471
1762	0.320546
1763	0.320605
1764	0.315527
1765	0.312879
1766	0.318468
1767	0.312204

Step	Training Loss
1768	0.294297
1769	0.314029
1770	0.322681
1771	0.320975
1772	0.316164
1773	0.332044
1774	0.314395
1775	0.324988
1776	0.303120
1777	0.317816
1778	0.322612
1779	0.327143
1780	0.332792
1781	0.319535
1782	0.323502
1783	0.327717
1784	0.325166
1785	0.313590
1786	0.321188
1787	0.315332
1788	0.317178
1789	0.319109
1790	0.331455
1791	0.324869
1792	0.332464
1793	0.320987
1794	0.332306
1795	0.316854
1796	0.317393
1797	0.314238
1798	0.312548
1799	0.311672
1800	0.305000
1801	0.308675

Step	Training Loss
1802	0.326717
1803	0.309199
1804	0.313248
1805	0.318796
1806	0.331280
1807	0.313244
1808	0.328106
1809	0.318384
1810	0.333262
1811	0.329291
1812	0.298874
1813	0.324478
1814	0.307127
1815	0.320288
1816	0.302263
1817	0.317981
1818	0.328432
1819	0.315494
1820	0.323292
1821	0.299314
1822	0.326888
1823	0.317352
1824	0.308925
1825	0.309788
1826	0.315455
1827	0.298920
1828	0.305909
1829	0.311866
1830	0.315391
1831	0.302688
1832	0.322480
1833	0.325662
1834	0.330380
1835	0.311730

Step	Training Loss
1836	0.307616
1837	0.309442
1838	0.326131
1839	0.314482
1840	0.325202
1841	0.329276
1842	0.330781
1843	0.307974
1844	0.328747
1845	0.326474
1846	0.317148
1847	0.322056
1848	0.322001
1849	0.313996
1850	0.304407
1851	0.318870
1852	0.328660
1853	0.318033
1854	0.305031
1855	0.319166
1856	0.319041
1857	0.321502
1858	0.327090
1859	0.310279
1860	0.320769
1861	0.303471
1862	0.317056
1863	0.327667
1864	0.324680
1865	0.322965
1866	0.326619
1867	0.322347
1868	0.311400
1869	0.326061

Step	Training Loss
1870	0.322282
1871	0.330656
1872	0.326397
1873	0.317333
1874	0.318939
1875	0.328568
1876	0.321231
1877	0.316618
1878	0.314283
1879	0.307480
1880	0.322119
1881	0.323557
1882	0.323602
1883	0.319349
1884	0.326227
1885	0.317444
1886	0.330048
1887	0.308957
1888	0.306638
1889	0.308109
1890	0.318313
1891	0.322631
1892	0.316626
1893	0.312046
1894	0.311659
1895	0.323684
1896	0.311442
1897	0.331955
1898	0.332880
1899	0.318485
1900	0.325922
1901	0.310203
1902	0.322974
1903	0.331568

Step	Training Loss
1904	0.325203
1905	0.305876
1906	0.321546
1907	0.314041
1908	0.330699
1909	0.320137
1910	0.309639
1911	0.327509
1912	0.316865
1913	0.337085
1914	0.314084
1915	0.311248
1916	0.317080
1917	0.299117
1918	0.311862
1919	0.316729
1920	0.299850
1921	0.332780
1922	0.319352
1923	0.313401
1924	0.310528
1925	0.293997
1926	0.324452
1927	0.312842
1928	0.303035
1929	0.306070
1930	0.344129
1931	0.310463
1932	0.305463
1933	0.338117
1934	0.344484
1935	0.307946
1936	0.321424
1937	0.321395

Step	Training Loss
1938	0.325842
1939	0.321682
1940	0.320056
1941	0.311200
1942	0.310260
1943	0.303820
1944	0.329204
1945	0.331313
1946	0.319960
1947	0.322489
1948	0.317162
1949	0.332855
1950	0.312906
1951	0.334902
1952	0.312075
1953	0.328121
1954	0.328573
1955	0.327387
1956	0.321634
1957	0.315593
1958	0.316482
1959	0.326245
1960	0.319943
1961	0.313277
1962	0.310824
1963	0.309899
1964	0.312540
1965	0.320740
1966	0.320016
1967	0.302204
1968	0.312188
1969	0.328626
1970	0.312216
1971	0.332203

Step	Training Loss
1972	0.314337
1973	0.331414
1974	0.327081
1975	0.311977
1976	0.327369
1977	0.326670
1978	0.307297
1979	0.313989
1980	0.312583
1981	0.313889
1982	0.319243
1983	0.327884
1984	0.322713
1985	0.314198
1986	0.327520
1987	0.324503
1988	0.315851
1989	0.321865
1990	0.319920
1991	0.305169
1992	0.313630
1993	0.325246
1994	0.310173
1995	0.308146
1996	0.327596
1997	0.337658
1998	0.313683
1999	0.327560
2000	0.327101

```
Training complete. LoRA adapters saved to: qwen2_5_vl_lora_fp16_t4
```

Questions to answer:

1. Report the settings you used to get the best model.

2. Which hyperparameters did you find have the most impact in the model's performance?

3. Why do you think that is?

See above for the best settings. Interestingly, my model stagnated early on (have run the above cell multiple times; it starts at about 7 in loss).

Likely the cross-entropy loss is not best---I should use a MSE loss and a continuous output variable. Learning rate is obviously the most impactful, though I found the 1e-4 to be best (nice standard value). I increased LoRA rank and batchsize to improve stability (achieved convergence much faster too), but did not get anything in accuracy.

One note; with smaller image sizes (shortest_edge) and less tokens for reasoning, the model performed quite badly. Since the images are mostly texture (vorticity shapes), the lower resolution seems to have destroyed performance.

Problem 6: Post-Training Evaluation (30 points)

Problem 6.1 Load the Trained LoRA Adapter

Once your fine-tuning is complete, load the trained **LoRA adapters** back onto the original model to perform inference, that is, to generate predictions or analyze new images.

Simply run the code in the next code block.
It will automatically attach your fine-tuned LoRA weights and prepare the model for evaluation.

In [49]:
```
# --------------------
# Inference with adapters
# --------------------
from peft import PeftModel
base = Qwen2_5_VLForConditionalGeneration.from_pretrained(
    MODEL_ID,
    dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="cuda:0",
    low_cpu_mem_usage=True,
    trust_remote_code=True,
)
ft_model = PeftModel.from_pretrained(base, OUTPUT_DIR)
ft_model.eval()
print("LoRA adapters loaded. Ready for inference.")
```

```
Fetching 2 files: 100%|██████████████████████████████████████████████████| 2/2 [00:
00<00:00, 14716.86it/s]
Loading weights: 100%|└└└
```

Compare the **pre-trained** (in Step 2.2) and **fine-tuned** model outputs:

- Which questions showed improvement?
- Did LoRA fine-tuning correct any earlier mistakes?
- Were any new errors or biases introduced after fine-tuning?

Document your observations and include examples where possible.

In [50]:
```
main(model=ft_model, SYSTEM_PROMPT="You are a helpful assistant.")
```

	image	response	true	time-error (in frames)	dimension error (log of ratio)	reasoning
0	No description has been provided for this image	196, 408	83, 42	113	2.273598	\n196, 408
1	No description has been provided for this image	196, 408	166, 16	30	3.238678	\n196, 408
2	No description has been provided for this image	196, 408	249, 260	-53	0.450586	\n196, 408
3	No description has been provided for this image	406, 189	332, 469	74	-0.908856	\n406, 189
4	No description has been provided for this image	194, 106	415, 418	-221	-1.372042	\n194, 106
5	No description has been provided for this image	406, 189	498, 10	-92	2.939162	\n406, 189

In [51]:
```
main(model=ft_model, SYSTEM_PROMPT="""We are analyzing quasigeostrophic turbulence.
Early times:
smooth flow with few vortices.
Middle times:
vortex merging and filament formation.
Late times:
many thin filaments and small vortices.
The flow eventually becomes self-similar and the intrinsic
manifold dimension saturates.
Use the visual structure of the field to estimate the stage
of turbulence; provide your best guess. Use previous images to inform.""", _build_chat_messages=l
```

	image	response	true	time-error (in frames)	dimension error (log of ratio)	reasoning
0	No description has been provided for this image	106, 11	83, 42	23	-1.339774	\n106, 11
1	No description has been provided for this image	280, 1237	166, 16	114	4.347856	\n280, 1237
2	No description has been provided for this image	295, 377	249, 260	46	0.371564	\n295, 377
3	No description has been provided for this image	206, 107	332, 469	-126	-1.477774	\n206, 107
4	No description has been provided for this image	306, 107	415, 418	-109	-1.362653	\n306, 107
5	No description has been provided for this image	395, 175	498, 10	-103	2.862201	\n395, 175

Actually much better --- the timestamps are closer now and we have a large but constant offset rather than a trend in the error. Even the dimensionality is closer (though for some reason the model greatly overestimates the complexity of the smooth field during spinup).

For some reason, the model still requires a couple few-shot examples...likely again because the task is so different than normal VLM tasks. My best guess is the examples help precondition the autoregressive latent.

Problem 7: Final Reflection (10 points)

Now we'll take some time to reflect on this homework. Take some time to discuss the following:

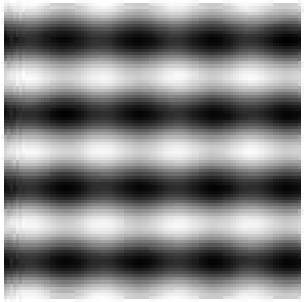
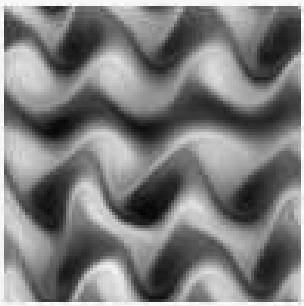
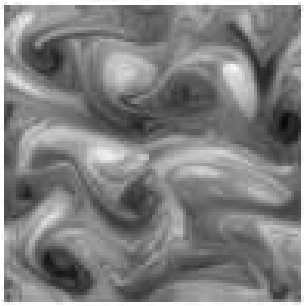
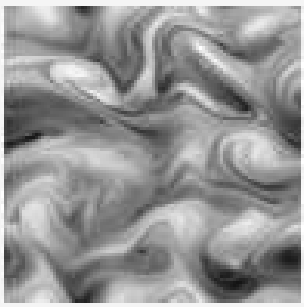
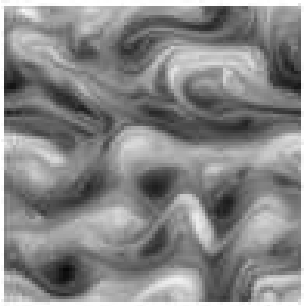
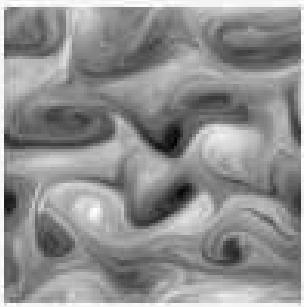
1. What concept did you find the most interesting?
2. Which concepts (if any) do you see being useful towards your goal? Why? If there was none, discuss why.
3. Is there a topic that was discussed during lectures up to the release of the assignment that you wished was covered in the homework? Any from the assignment that you wanted there to be touched upon more?

Ans:

1. I liked the questions at the beginning (though a bit open-ended with respect to the readings). I have seen a lot of recent discussion about distribution separation across my classes, so it was nice to tie them together a little.
2. Not much, unfortunately. I had high hopes for the LLM: actually, there are papers that tokenize fluid flow and use an LLM to predict it, so it could have gone quite well. Basically, this assignment helped confirm that the standard VLMs aren't quite enough for complex fluid flows, because of the amount of difficulty they had with some basic questions (that are needed to even set up the prediction or generation process).
3. Probably the architecture. A lot of these models (like this VLM) are essentially a black-box as far as the assignment and multimodal aspects are concerned---which is definitely not the case for practical problems. There's still too much a disconnect with practice for me. I would like to understand these models at the level of the attention head, which is actually doable for most smaller models. Maybe we

could do an assignment where we inspect the saliency maps of a VLM or something and see where the fusion happens, how, and why... (like attention heat maps, or even the famous CNN visualization paper)?

In [ ]:

0		1
1		1
2		2
3		3
4		2
5		2